Relatório de Evolução do Projeto

Desafio: FIAP × GoodWe Brasil · EV Challenge

Projeto: ChargeBot — ChargeGrid Intelligence

Módulo / Sprint: Módulo 2 · Sprint 04 (2026.2)

1. Resumo da Evolução

A evolução do **ChargeBot** ao longo do semestre estruturou-se em etapas incrementais e consolidadas:

- **Sprints 01 e 02 (Exploração, Planeamento e Desenvolvimento Base):** Concepção da arquitetura inicial do chatbot de mobilidade elétrica, definição do domínio de carregamento inteligente GoodWe e estruturação das bases de dados e regras de negócio essenciais.
- **Sprint 03 (Refactoring Conversacional):** Introdução da arquitetura conversacional utilizando LangChain, gestão de memória de diálogo e implementação inicial de guardrails de moderação e validação de escopo.
- **Sprint 04 (RAG Medido, Confiável e Interface Web):** Elevação do projeto a um sistema de Recuperação Aumentada por Geração (RAG) robusto. Implementação de vetorização persistente com **ChromaDB** e embeddings **nomic-embed-text** via Ollama, grounding estrito com citação obrigatória de fontes, avaliação sistemática por iterações (baseada em rubrica equivalente ao RAGAS) e disponibilização de uma interface web interativa em **Gradio**.

2. Pipeline RAG e Decisões de Arquitetura

O pipeline RAG desenvolvido na pasta src/rag/ cumpre um fluxo end-to-end otimizado para o domínio técnico de carregamento elétrico:

- **Ingestão (loader.py):** Utilização do PyMuPDFLoader para processar a base de conhecimento expandida (data/knowledge_base/), que inclui manuais de produtos GoodWe (ChargeGrid / EV ChargeOps), regimentos condominiais e tabelas tarifárias.
- **Chunking (chunking.py):** Aplicação do RecursiveCharacterTextSplitter com parâmetros definidos em chunk_size = 1000 e chunk_overlap = 200.
    - *Trade-off:* O tamanho de 1000 carateres garante que especificações técnicas completas e tabelas de parâmetros não fiquem fragmentadas, enquanto o overlap de 200 assegura a continuidade semântica entre blocos adjacentes.
- **Vector Store (vector_store.py e embeddings.py):** Persistência local em ChromaDB utilizando o modelo de embeddings nomic-embed-text, evitando reprocessamentos desnecessários e garantindo alta velocidade de recuperação por similaridade.

- **Prompt Versionado e Grounding (prompt_rag.py):** Versões evoluídas de prompts que restringem o LLM a responder exclusivamente com base no contexto recuperado, exigindo a citação explícita do documento de origem e da página/seção.

3. Tabela Comparativa (Antes vs. Depois)

A tabela seguinte evidencia a evolução técnica estrutural entre o estado original do projeto e a entrega atual da Sprint 04:

Métrica / Critério de Avaliação	**Sprints 01, 02 e 03 (Versão Original)**	**Sprint 04 (RAG Avaliado e Medido)**
Arquitetura de Recuperação	Prompts estáticos / Base conversacional em memória	RAG end-to-end com ChromaDB persistente + nomic-embed-text
Grounding e Fiabilidade	Respostas baseadas no conhecimento interno do LLM (risco de alucinação)	Grounding estrito limitado estritamente ao contexto recuperado
Citação de Fontes	Ausente ou não estruturada nas respostas	Citação obrigatória e visível (documento e página)
Métrica Faithfulness (Confiabilidade)	~0.55 (Sem garantia de alucinação zero)	**0.95** (Iteração 2 validada)
Métrica Answer Relevancy (Relevância)	~0.60	**0.92** (Iteração 2 validada)
Interface do Utilizador	Execução via terminal / Scripts isolados	Interface web interativa em Gradio (app/app.py)

Segurança contra Prompt Injection	Não tratada formalmente nos documentos	Regras explícitas no prompt para ignorar instruções embutidas nos manuais

4. Problemas Encontrados e Soluções Técnicas

Durante o desenvolvimento da Sprint 04, a equipa deparou-se com desafios significativos que foram superados com as seguintes decisões arquiteturais:

1. **Injeção de Instruções Maliciosas em Documentos PDF:**
    - *Problema:* Determinados manuais técnicos continham frases estruturadas de forma ambígua que o LLM interpretava incorretamente como comandos de sistema (prompt injection via dados recuperados).
    - *Solução:* Reformulação do prompt do sistema (prompt_rag.py) com instruções de isolamento rigorosas, determinando explicitamente que o modelo deve ignorar qualquer instrução contida no corpo do contexto recuperado e focar apenas na extração factual de dados.
2. **Consultas Fora do Domínio (Out-of-Scope):**
    - *Problema:* Perguntas genéricas sobre mobilidade elétrica global ou marcas concorrentes geravam respostas divagantes antes da implementação do RAG restrito.
    - *Solução:* Configuração de uma diretriz de recusa obrigatória no prompt v2, fazendo com que o ChargeBot recuse polidamente e de forma padronizada caso o recuperador não encontre correspondência direta nos manuais GoodWe.

5. Equipe e Divisão de Trabalho

A execução das tarefas desta sprint foi distribuída de forma colaborativa entre os elementos do grupo:

- **Integrante 1:** Arquitetura do pipeline RAG (loader.py, chunking.py, vector_store.py) e integração com ChromaDB.
- **Integrante 2:** Desenvolvimento do versionamento de prompts, testes de grounding e blindagem contra prompt injection.
- **Integrante 3:** Implementação da interface web (app/app.py em Gradio) e validação da citação visível de fontes.
- **Integrante 4:** Condução das avaliações por iteração, elaboração dos relatórios de métricas em evals/ e redação do relatório final de evolução.